CryptoLens

Phase 2 Deliverables Document

Bronze & Silver Lakehouse Implementation

Field	Detail
Team Members	Abdullah Saeed (24L-2529); Arqum Umais (24L-2591)
GitHub Repository	https://github.com/x-abdullah-saeed-x/CryptoLens
Platform	Databricks Free Edition (Serverless) — Apache Spark, Delta Lake, Unity Catalog
Document Scope	Phase 2: Bronze + Silver pipeline, schema enforcement, idempotency, audit logging

1. Overview

Phase 2 transitions CryptoLens from a one-off download script into an enterprise-pattern PySpark pipeline. This document maps each Phase 2 requirement to its concrete implementation in 01.2_bronze_ingest.ipynb and 02_silver_transform.ipynb (supported by 01.1_bronze_ingest_data_extractor.ipynb for local data acquisition and 00_layers_verification.ipynb for validation), all committed to the GitHub repository.

2. Infrastructure & Environment

- Workspace: Databricks Free Edition (Serverless SQL/Compute), Unity Catalog enabled.
- Storage: raw files land in a Unity Catalog Volume (workspace.cryptolens.bronze_raw); all structured tables (Bronze, Silver, dim_symbol, audit logs) are managed Delta tables in the workspace.cryptolens schema.
- Version control: all notebooks, this document, and the data dictionary are committed to the GitHub repository; Databricks Git folders keep the workspace copy in sync via commit/push.
- Environment constraint: Free Edition's serverless compute restricts outbound internet access to an internal allowlist that excludes Binance's domains (confirmed via direct DNS resolution testing — api.binance.com and data.binance.vision fail to resolve, while pypi.org and github.com succeed). Raw file acquisition therefore runs on a local machine and verified files are pushed into the Volume via the Databricks CLI (databricks fs cp).

3. Data Modeling & Schema Enforcement

Full column-level definitions are provided in the separate Data Dictionary (CryptoLens_Data_Dictionary.pdf). Summary of how each requirement is met:

Requirement	Implementation
No inferSchema	RAW_KLINE_SCHEMA and LOG_SCHEMA are explicit StructType/StructField definitions in both notebooks; every .csv() read passes .schema(...) explicitly.
Strict data types & casting	Bronze keeps Binance's native LONG/DOUBLE types. Silver casts open_time/close_time (epoch ms) to TIMESTAMP, price fields (open/high/low/close) to DECIMAL(18,8), volume fields (volume, quote_asset_volume, taker_buy_base_vol, taker_buy_quote_vol) to the wider DECIMAL(28,8), and number_of_trades to INTEGER.
load_timestamp on every record	Added via current_timestamp() on every write, in both bronze_klines/bronze_exchange_info and silver_klines/dim_symbol.

4. Pipeline Robustness & Idempotency

4.1 Idempotent execution (MERGE INTO)

- Bronze: MERGE on (symbol, interval, load_type, open_time) with WHEN NOT MATCHED THEN INSERT — re-ingesting the same file inserts zero duplicate rows.

- Silver: MERGE on (symbol, interval, open_time) with WHEN MATCHED THEN UPDATE / WHEN NOT MATCHED THEN INSERT — also correctly handles a later full-load file overwriting a value from an earlier daily incremental file for the same timestamp.
- dim_symbol: MERGE with three branches — WHEN MATCHED (status/filter changed) THEN UPDATE, WHEN NOT MATCHED BY TARGET THEN INSERT (new listing), WHEN NOT MATCHED BY SOURCE THEN soft-delete (is_active = false, status = 'DELISTED').

4.2 Parameterized backfills

The Bronze notebook exposes Databricks widgets for interval, load_type, year, month, and day, and builds the Volume file path dynamically from them (build_volume_path()). Symbols are deliberately NOT a manual parameter: discover_symbols() scans the Volume and processes every symbol that has a file for the chosen date/load_type, so there is no hand-maintained symbol list to drift out of sync. The Silver notebook takes interval and load_type and discovers symbols from the distinct values present in bronze_klines. No function assumes "today"; any historical month/day can be re-processed by changing widget values and re-running.

4.3 Schema drift handling

- CSV reads use mode='PERMISSIVE' with columnNameOfCorruptRecord='_corrupt_record' — rows that don't match RAW_KLINE_SCHEMA populate this column instead of failing the batch.
- Rows with a populated _corrupt_record are written to bronze_quarantine (with option("mergeSchema","true") so the quarantine table itself can tolerate evolving bad-row shapes) rather than crashing the pipeline.
- Delta writes to Bronze use mergeSchema=true to gracefully evolve the table if Binance adds a column in the future.

5. Audit & Execution Logging

pipeline_execution_logs (Delta table, schema in the Data Dictionary) receives one row from every call to log_execution(), on every success, failure, and skip path in both notebooks. Logged fields: layer (Raw-to-Bronze / Bronze-to-Silver), the file path or symbol-interval-loadtype parameter processed, load_type, start_time, end_time, status, rows_inserted, rows_updated, and error_message.

This means every file processed across both Full and Incremental loads, at both layers, leaves an auditable trace — queryable directly as a Delta table for monitoring or troubleshooting.

6. Verification (00_layers_verification.ipynb)

A dedicated verification notebook is committed alongside the pipeline so every claim above can be re-checked on demand rather than taken on trust. It performs the following checks:

- Table existence: confirms bronze_klines, silver_klines, pipeline_execution_logs (and bronze_quarantine / dim_symbol / bronze_exchange_info where applicable) exist in workspace.cryptolens.
- Row and symbol counts per layer: compares Bronze and Silver row counts and distinct-symbol counts. Silver is expected to hold slightly fewer rows (quality rules drop invalid candles) but the same symbols.
- Idempotency: records the Silver row count, re-runs the Silver driver loop with identical parameters, and confirms the count is unchanged (MERGE inserted zero duplicates).
- Data sanity spot-check: inspects sample rows for a known symbol (e.g. BTCUSDT) to confirm timestamps are real TIMESTAMP values and prices/volumes are sensible.
- Audit trail: reviews pipeline_execution_logs for a SUCCESS row per processed file, no unexpected FAILURE rows, and non-zero row counts.
- Quarantine and quality filters: counts rows in bronze_quarantine and the number of rows dropped between Bronze and Silver, confirming the validation rules are actually active.

7. Issues Encountered and Resolved

Running the pipeline against real top-50 data on Serverless compute surfaced several issues. They are documented here because each reflects a genuine platform or data constraint, not just a typo.

Issue	Root cause	Resolution
Outbound download blocked (NameResolutionError for api.binance.com)	Free Edition serverless compute enforces a domain allowlist excluding Binance; confirmed by DNS test (pypi.org/github.com resolve, Binance does not).	Download locally (01.1_bronze_ingest_data_extractor.ipynb), verify SHA256 checksums, upload to the Volume via Databricks CLI.
Empty Bronze table after first run	Driver loop depended on dim_symbol existing to resolve symbol 'ALL'; on a first run it did not exist, producing an empty symbol list and a silent no-op.	Removed the manual symbol widget; discover_symbols() scans the Volume directly, and an explicit warning prints if zero files match.
RDD_NOT_SUPPORTED in Silver	DataFrame.isEmpty() wraps .rdd.isEmpty() internally; RDDs are unavailable on Serverless compute.	Replaced with limit(1).count() == 0, which stays on the DataFrame/SQL execution path.
NUMERIC_VALUE_OUT_OF_RANGE casting volume to DECIMAL(18,8)	Low-priced, high-supply symbols trade volumes exceeding the 10 integer digits allowed by DECIMAL(18,8).	Widened volume-type columns to DECIMAL(28,8); price columns unchanged.
DELTA_CAST_OVERFLOW_IN_TABLE_WRITE on MERGE	silver_klines had been created earlier with the old DECIMAL(18,8) schema; Delta locks column types at creation, so wider values could not be merged in.	Dropped silver_klines during development and let the first write recreate it with the corrected schema (mergeSchema does not widen existing column types).

8. Repository Deliverables

- notebooks/00_layers_verification.ipynb — validation queries described in Section 6.
- notebooks/01.1_bronze_ingest_data_extractor.ipynb — local-machine acquisition: download, SHA256 verification, extraction.
- notebooks/01.2_bronze_ingest.ipynb — Raw-to-Bronze Databricks pipeline.
- notebooks/02_silver_transform.ipynb — Bronze-to-Silver Databricks pipeline.
- docs/CryptoLens_Data_Dictionary.pdf — column-level definitions for every table.
- docs/Phase2_CryptoLens.pdf — this document.

9. Known Limitations / Next Steps (Phase 3)

- Gold layer (fact_daily_ohlcv, volatility, moving averages, correlation matrix) is designed but not yet implemented — scheduled for Phase 3 alongside the Power BI dashboard.
- exchangeInfo snapshot ingestion requires a JSON file to be manually uploaded to the Volume via the same local-download workaround; automating it into a recurring job is future work.
- If silver_klines is ever to be retained across schema changes in production, Delta type-widening (delta.enableTypeWidening) should replace drop-and-rebuild.